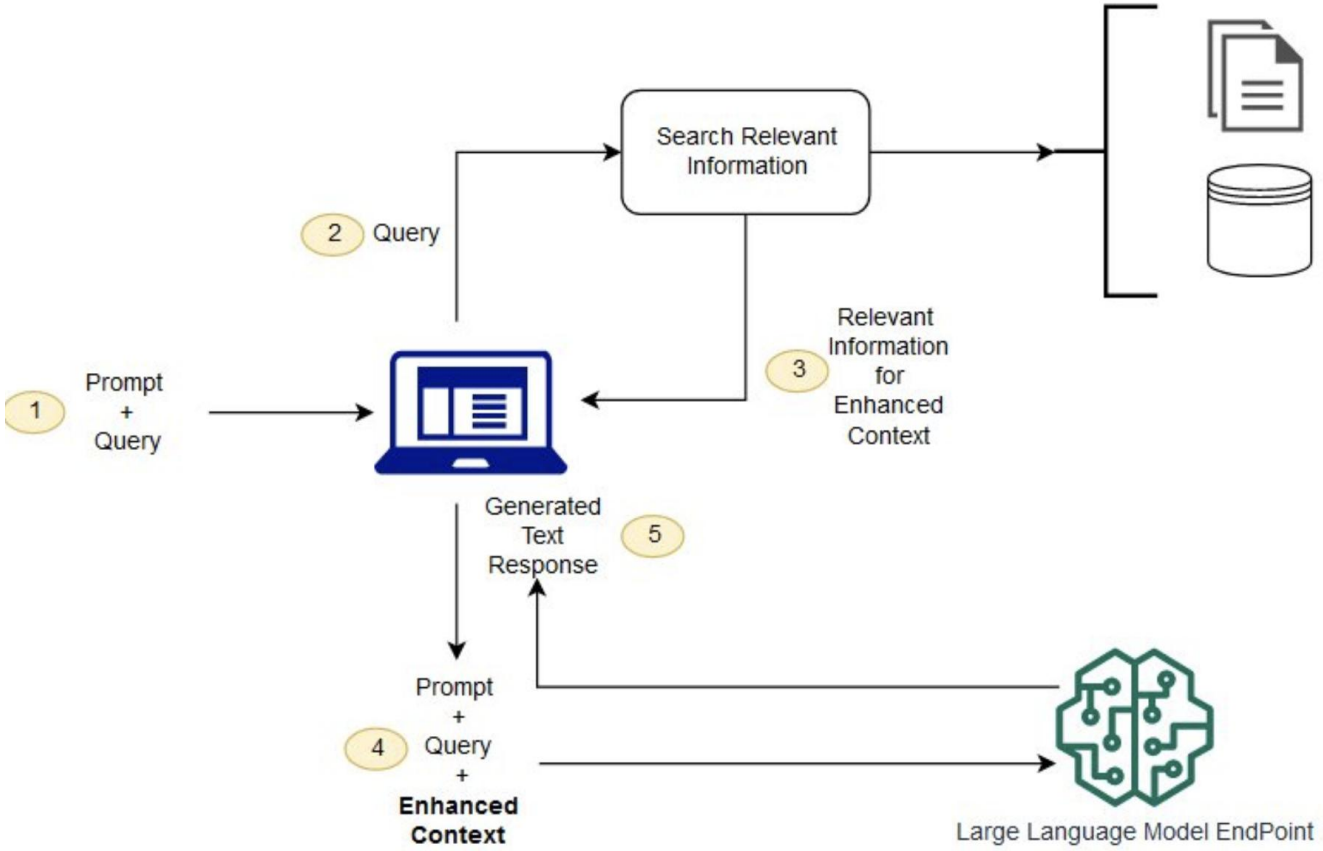
Retrieval-Augmented Generation (RAG) in AIWare Software Projects

Saeed Siddik
Assistant Professor
IIT University of Dhaka

What is RAG?

- Retrieval-Augmented Generation (RAG) is an AI architecture that combines an information retrieval system with a language model
- Responses are generated using both the model's knowledge and externally retrieved, up-to-date data.
- RAG enables AI-enabled software products to be more accurate, maintainable, and adaptable by separating knowledge management (indexing and retrieval) from language understanding and response creation (generation).

Flow of RAG System



How RAG system works?

Indexing stage

- knowledge sources such as documents, books, databases, or policies are collected, cleaned, split into smaller chunks,
- converted into vector embeddings,
- stored in a searchable index (usually a vector database),
- prepares the knowledge for efficient access.

Retrieval stage

- when a user asks a question, the system converts the query into an embedding
- searches the index to find the most relevant pieces of information,
- often filtering or re-ranking them to improve relevance and accuracy.

Generation stage

- the retrieved content is injected into the prompt of a language model,
- then generates a response that is grounded in the retrieved evidence,
- reducing hallucination and increasing trustworthiness.

RAQ vs Fine Tuning

Aspect	RAG	Fine-Tuning
Basic Idea	Retrieves external knowledge at query time and uses it during generation	Embeds knowledge directly into the model's weights
Knowledge Update	Easy to update by change/add documents	Requires retraining / re-fine-tuning the model
Data Freshness	Can use real-time or frequently updated data	Limited to data available during training
Hallucination Risk	Lower, as answers are grounded in sources	Higher, since model relies on internal memory
Explainability	High, retrieved documents can be shown as evidence	Low, model decisions are not easily traceable
Scalability	Scales well by expanding the knowledge	Poor scalability as data size grows
Latency	Slightly higher due to retrieval step	Lower at inference time
Best Use Cases	Enterprise search, QA systems, policy assistants	Style adaptation, domain language learning
Software Engineering Fit	Modular and aligns with modern software architecture	Tightly coupled to the model
AIWare Projects	Yes, for most production systems	Only for narrow, stable domains

Query translation approaches in RAG

- techniques used to reformulate or enrich a user's original query so that the retrieval component can fetch more relevant and useful information before generation.

Multi-query translation

- the original query is rewritten into multiple semantically similar queries.
- each rewritten query captures a different phrasing or interpretation of the same intent,
- retrieval is performed for all of them.
- increases recall and reduces the risk of missing relevant documents due to wording differences.

RAG-fusion

- builds on the multi-query idea by combining and re-ranking the retrieved results from all rewritten queries
- instead of treating each retrieval independently, RAG-fusion merges results using ranking or score aggregation techniques, prioritizing documents
- appear consistently across multiple queries, which helps balance diversity with relevance.

Least-to-most query translation

- In least-to-most query translation, a complex question is decomposed into a sequence of simpler sub-queries.
- Retrieval is first performed for basic concepts, and the results are then used to support retrieval for more complex reasoning steps.
- This approach is particularly useful for multi-step or analytical questions.

Query routing strategies

- Query routing strategies in RAG determine where and how a user query should be processed before retrieval and generation.
- Instead of sending every query to the same knowledge source

Logical routing

- Logical routing relies on explicit rules, conditions, or metadata to decide how a query should be handled.
- The system uses predefined logic such as keywords, query structure, user role, domain tags, or access permissions to route the query to a specific knowledge source or retrieval pipeline.
- For example, queries containing policy-related keywords may be routed to a compliance document store, while technical queries are sent to a developer knowledge base.

Semantic routing

- Semantic routing uses embeddings and similarity-based reasoning to route queries based on their meaning rather than explicit rules.
- The user query is embedded and compared against representations of different domains, tools, or knowledge sources, and the system routes the query to the most semantically relevant pipeline.

Types of RAG

- Standard (Vector-Based) RAG
  - This is the most common form of RAG, where documents are embedded as vectors and stored in a vector database.
  - Retrieval is performed using semantic similarity search, and the retrieved text is passed to the language model for generation. It is widely used in document QA and enterprise search systems.

Types of RAG

- Graph RAG
  - Graph RAG represents knowledge as entities and relationships stored in a knowledge graph.
  - Instead of retrieving isolated text chunks, the system traverses graph connections to collect structured, context-rich information.
  - This type of RAG is well-suited for complex reasoning, multi-hop questions, and domains with well-defined relationships such as finance, healthcare, or supply chains.

Types of RAG

- Conversational RAG
  - Conversational RAG maintains dialogue history and user context across multiple turns.
  - Retrieval is influenced by previous questions and answers, allowing the system to provide coherent and context-aware responses in chat-based applications.

Types of RAG

- Multi-Source RAG
  - This type of RAG retrieves knowledge from multiple sources such as documents, databases, APIs, and knowledge bases.
  - The retrieved information is aggregated and normalized before generation.
  - Multi-source RAG is common in enterprise assistants and decision-support systems.

16 TYPES OF RAG

Type	Key Features	Benefits	Applications / Requirements	Examples of Tooling/Library
Standard RAG (RAG-Sequence and RAG-Token)	• Basic retrieval and generation integration • RAG-Sequence and RAG-Token variants	• Improved accuracy • Reduced hallucinations	• General-purpose QA systems • Initial RAG implementations	Hugging Face Transformers Facebook's RAG Implementation LangChain
Agentic RAG	• Autonomous agents • Tool use • Dynamic retrieval	• Handles complex tasks • Proactive AI	• Personal Assistants • Research aids • Customer service bots needing dynamic interaction	LangChain Agents OpenAI GPT-4 with Plugins Microsoft'Semantic Kernal
Graph RAG	• Knowledge graphs • Relational reasoning	• Rich information • Context handling	• Expert systems in medicine, law, engineering • Semantic search engines	Neo4j Graph Database Apache Jena Stardog
Modular RAG	• Independent modules for retrieval, reasoning, generation	• Flexibility • Scalability	• Large projects requiring collaborative development • Systems needing frequent updates	Microservices Architecture Docker & Kubernetes Apache Kafka
Memory-Augmented RAG	• External memory storage and retrieval	• Continuity • Personalization	• Chatbots maintaining long-term context • Personalized recommendations	Redis for Session Storage Amazon Dynamo DB Pinecone Vector Database
Multi-Modal RAG	• Cross – modal retrieval (text, images, audio)	• Richer response • Accessibility	• Image captioning • Video Summarization • Multi-modal assistants	OpenAI's CLIP TensorFlow Hub Models PyTorch Multi-Modal Libraries
Federated RAG	• Decentralized data sources • Privacy-preserving	• Data security • Compliance	• Healthcare systems handling sensitive data • Collaborative platforms across organizations	TensorFlow Federated PySyft by OpenMinded • Federated Learning Libraries
Streaming RAG	• Real-time data retrieval and generated	• Up-to-date information • Low latency	• Live reporting • Financial tickers • Social media monitoring	Apache Kafka Streams Amazon Kinesis • Stark Streaming
ODQA RAG (Open-Domain Question Answering)	• Broad knowledge base • Dynamic retrieval	• Broad applicability • Dynamic responses	• Search engines • Virtual assistants handling diverse queries	Elasticsearch Haystack by Deepset Hugging Face Transformers
Contextual Retrieval RAG	• Context-aware retrieval using conversation history	• Personalization • Coherence	• Conversational AI • Customer support chatbots maintaining session-context	Dialogflow by Google Rasa Open Source Microsoft Bot Framework
Knowledge-Enhanced RAG	• Integration of structured knowledge bases	• Factual accuracy • Domain expertise	• Educational tools • Professional domain applications (legal, medical)	Knowledge Graph Embeddings Libraries OWL API Apache Jena
Domain-Specific RAG	• Customized for specific industries or fields	• Relevance • Compliance • Trustworthiness	• Legal research assistants • medical diagnosis support • Financial analysis tools	• LexPredict Contract Analytics Watson Health • Financial NLP Tools
Hybrid RAG	• Combining multiple retrieval approaches	• Improved recall • Enhanced relevance	• Complex QA systems • Search engines needing both lexical and semantic matching	Elasticsearch with kNN Plugin FAISS by Facebook AI • Hybrid Retrieval Libraries
Self-RAG	• Self-reflection mechanisms • iterative refinement	• Enhanced accuracy • Improved coherence	• Content creation tools • Educational platforms requiring high accuracy	OpenAI GPT Models with Fine-Tuning • Human-in-the-Loop Platforms
HyDE RAG (Hypothetical Document Embeddings)	• Hypothetical document embeddings for guided retrieval	• Better recall • Improved answer quality	• Complex queries with implicit meanings • Research assistance in niche fields	• Custom Implementations with Transformers Haystack Pipelines
Recursive / Multi-Step RAG	• Multiple rounds of retrieval and generation	• Enhanced reasoning • Greater understanding	• Analytical and problem-solving tasks • Dialogue systems with multi-turn interactions	LangChain's Chains and Agents DeepMind's AlphaCode Framework

End of RAG